

Detección de Clientes: Suscripción de Depósito Bancario

Facundo Casas

facundocasas@uca.edu.ar

Universidad Católica Argentina

Javier Balda

javierbalda@uca.edu.ar

Universidad Católica Argentina

Juan Caracoix

juancaracoix@uca.edu.ar

Universidad Católica Argentina

ABSTRACT

Presentamos un estudio y una solución práctica para la predicción de la suscripción de clientes a depósitos bancarios. El proyecto combina un pipeline de preprocesado, balanceo de clases (SMOTE), entrenamiento de modelos de clasificación (implementaciones con XGBoost) y un componente de explicabilidad denominado COLOSSUS que integra similitud del coseno sobre parámetros y lógica simbólica para extraer reglas interpretable. La solución incluye: (i) un pipeline reproducible para entrenamiento y evaluación, (ii) una API de inferencia basada en FastAPI para despliegue, y (iii) un dashboard de seguimiento (Streamlit / MLflow) para monitorizar métricas como precision, recall y ROC AUC. En los experimentos presentados, el modelo alcanza alta discriminación (ROC AUC 0.95) y mejora estimada en ROI al focalizar campañas sobre segmentos de alta probabilidad.

Palabras clave: Inteligencia Artificial, Modelos de Caja Negra, Similitud del Coseno, Aprendizaje Profundo, Explicabilidad, Redes Neuronales, Lógica, Regularización, Extracción de Reglas

1. INTRODUCCIÓN

Las campañas de marketing directo en el sector bancario (llamadas telefónicas para ofrecer depósitos a plazo) suponen un coste operativo elevado cuando se aplican de forma indiscriminada. El objetivo de este trabajo es identificar clientes con mayor probabilidad de aceptar la oferta, optimizar el targeting de las campañas y reducir llamadas innecesarias, mejorando al mismo tiempo el ROI. Para ello se utiliza el dataset de Bank Marketing (UCI) junto con fuentes internas que contienen información demográfica, financiera y de comportamiento del cliente.

El proyecto implementa un flujo completo que va desde la ingesta y preprocesado hasta el despliegue: limpieza y codificación de variables categóricas, tratamiento de outliers, balanceo de clases con SMOTE, entrenamiento de modelos de clasificación (XGBoost en la implementación de referencia), evaluación mediante métricas estándar (precision, recall, f1, ROC AUC) y despliegue de la inferencia mediante una API y un dashboard de seguimiento.

2. EXPLICABILIDAD

La explicabilidad es clave para confiar en decisiones automatizadas que afectan a clientes reales y para cumplir requisitos regulatorios. En este trabajo perseguimos explicaciones que sean:

- **Fieles:** reflejen el comportamiento real del modelo.
- **Comprensibles:** expresadas en reglas o condiciones que un analista o gestor de campaña pueda interpretar.
- **Estables:** no sensibles a pequeñas perturbaciones en los datos o en la semilla de entrenamiento.

Para alcanzar estos objetivos proponemos un enfoque híbrido (COLOSSUS) que combina medidas geométricas sobre parámetros del modelo (similitud del coseno entre vectores de pesos/capas) con técnicas de extracción simbólica de reglas, priorizando reglas con baja entropía y alta consistencia entre instancias similares.

3. IMPLEMENTACIÓN Y DESPLIEGUE

En esta sección se documentan los componentes de software desarrollados para entrenar, servir y procesar los datos del modelo, basados en la implementación disponible en la carpeta `scr/` del repositorio.

3.1. Modelo y artefactos

El modelo entrenado se empaqueta como un pipeline serializado en `scr/app/model/trained_pipeline-` y su versión explícita en el código es 0.1.0. La interfaz de inferencia expone dos funciones principales en `scr.app.model.model`:

- `predict_pipeline(input_data)`: devuelve la clase predicha (0/1).
- `predict_pipeline_proba(input_data)`: devuelve la clase y las probabilidades por clase.

La lista de variables esperadas por el pipeline (`FEATURE_COLUMNS`) incluye columnas numéricas y variables dummificadas. Es crítico que los datos de entrada respeten este orden y la presencia de columnas; en caso de discrepancia el servicio devuelve errores indicando las columnas faltantes.

3.2. API y Endpoints

La aplicación web se implementa con FastAPI en `scr/app/main.py` y en `scr/app/routes`. Los endpoints relevantes documentados en el código son:

- GET `/`: información básica y versión del modelo.
- GET `/ping`: comprobación simple (pong).
- GET `/healthz`: health check.
- GET `/readyz`: readiness (verifica si el modelo está disponible).
- GET `/info`: información del servicio y entorno.
- POST `/predict` y POST `/predict/single`: endpoints de inferencia que aceptan el esquema `ClientData` (Pydantic) y devuelven la predicción y probabilidades.

El código incluye manejo de errores centralizado mediante el decorador `handle_exceptions` definido en `scr/utils/errors.py`, que traduce excepciones internas a respuestas HTTP apropiadas.

3.3. Pipeline de Entrenamiento

El flujo de entrenamiento está definido en `scr/training/train_pipeline.py` y utiliza Prefect para orquestación. Los pasos principales son:

- Carga de datos desde una base MySQL mediante SQLAlchemy y guardado temporal en parquet.

- Aplicación de oversampling (SMOTE) para balancear la variable objetivo (en la implementación se fuerza un mínimo por clase, por defecto se apunta a 10.000 ejemplos para la clase minoritaria en algunos pasos).
- Guardado de los datos transformados en una tabla MySQL preparada (esquema definido en `scr/training/esquema_DB_train.py`).
- Limpieza de archivos temporales en `/tmp/bancox_train`.

El uso de SMOTE y el número objetivo por clase son decisiones de diseño que deben ser justificadas y validadas experimentalmente.

3.4. Procesamiento y Preprocesado

El módulo de procesamiento principal es una clase abstracta `Processor` definida en `scr/processing/processor.py`, con una implementación de ejemplo `ETLProcessor` que realiza validaciones básicas (comprobación de columnas esperadas), registro de columnas faltantes y limpieza ligera (eliminación de duplicados). Esta abstracción facilita crear pasos de ETL reproducibles y con logging estandarizado.

3.5. Registro y Monitoreo

El proyecto incluye un wrapper de logging en `scr/utils/log.py` que configura logging con un `RotatingFileHandler` y soporte para configurar fichero y nivel mediante variables de entorno (`LOG_FILE`, `LOG_LEVEL`). Prefect se utiliza para telemetría adicional en flujos y tareas asíncronas.

3.6. Requisitos y Ejecución

El servicio web contiene un `requirements.txt` en `scr/app/` que lista dependencias de la API. El pipeline de entrenamiento depende de paquetes adicionales como `pymysql`, `sqlalchemy`, `imblearn` y `prefect`. Para ejecutar localmente:

- Iniciar la API con Uvicorn apuntando a `scr.app.main:app`.
- Ejecutar el flow de entrenamiento mediante Prefect o ejecutando directamente `scr/training/train_` (requiere variables de entorno para MySQL).

4. ARQUITECTURA DEL SISTEMA

La arquitectura del sistema integra varios componentes diseñados para separar responsabilidades y facilitar despliegue y mantenimiento. A continuación se describe de forma textual cada componente y los flujos de datos principales:

Flujo general. El cliente (aplicación frontend o consumidor de la API) realiza peticiones HTTP a la API REST implementada con FastAPI. La API valida el esquema de entrada, transforma los datos según sea necesario y delega la inferencia al artefacto del modelo. Para operaciones de entrenamiento o tareas asíncronas se utiliza un orquestador (Prefect) que coordina la carga de datos, preprocesado, oversampling y almacenamiento de artefactos.

Componentes principales.

- **Cliente / Aplicación Frontend:** realiza solicitudes de inferencia y de estado (health/readiness). Debe validar a nivel de UI los campos obligatorios antes de enviar la petición.
- **API REST (FastAPI, Uvicorn):** punto de entrada que valida esquemas (Pydantic), aplica autenticación y autorizaciones si corresponde, enruta a la lógica de inferencia y a endpoints de diagnóstico. Implementa manejo centralizado de errores y registra eventos en el sistema de logging.
- **Modelo (pipeline serializado):** pipeline serializado que incluye transformaciones y estimador final. Se carga por la API para realizar predicciones mediante funciones como `predict_pipeline` y `predict_pipeline_proba`. El pipeline espera un conjunto fijo de features en orden y tamaño determinados.
- **Orquestador (Prefect):** coordina flujos de entrenamiento y tareas programadas (reentrenamientos, exportación de métricas, limpieza). Prefect ejecuta tareas que leen/escriben en la base de datos y en el almacenamiento temporal, y registra telemetría de ejecución.
- **Base de Datos (MySQL):** almacena tablas con datos crudos y tablas preparadas para entrenamiento. Sirve como fuente de verdad para historiales y auditoría.
- **Almacenamiento Temporal (/tmp, Parquet):** usado para intercambiar artefactos intermedios (por ejemplo archivos parquet) entre etapas del pipeline sin recargar la base de datos en cada paso.
- **Logging:** sistema de logging centralizado (archivos rotativos y/o stdout) que recoge eventos, errores y métricas operativas. Permite configurar la ruta y el nivel por medio de variables de entorno.

Relaciones y dependencias. La API y Prefect son consumidores directos del modelo y la base de datos; Prefect orquesta tareas que generan artefactos que luego pueden ser consultados por la API o almacenados en la DB. El almacenamiento temporal actúa como buffer entre pasos pesados de I/O. El logging recoge eventos de todos los componentes para auditoría y debugging.

4.1. Leyenda y responsabilidades

En el apartado siguiente se describen brevemente las responsabilidades de cada componente (equivalente a una leyenda textual del diagrama eliminado):

- **Cliente / Aplicación Frontend:** envía peticiones HTTP al API para solicitar inferencias o consultar el estado del servicio. Implementa validaciones básicas de entrada y consume las respuestas JSON.
- **API REST (FastAPI, Uvicorn):** punto de entrada para solicitudes externas. Valida esquemas (Pydantic), enruta llamadas a funciones de inferencia, aplica manejo centralizado de errores y expone endpoints de health/readiness. Registra eventos relevantes en el sistema de logging.
- **Modelo (pipeline serializado):** artefacto serializado que contiene transformaciones y estimador final. Se carga en memoria por la API para realizar predicciones mediante funciones como `predict_pipeline` y `predict_pipeline_proba`. Debe mantenerse versionado y consistente con la lista de features esperadas.
- **Orquestador (Prefect):** coordina flujos de entrenamiento e inferencia asíncrona, registra telemetría de ejecución y ejecuta tareas programadas (por ejemplo, reentrenamientos). Maneja persistencia intermedia y notificaciones dentro del pipeline.
- **Base de Datos (MySQL):** almacena las tablas de datos crudos y los datos preparados resultantes del proceso de ETL/training. Es la fuente de verdad para los datasets usados en entrenamientos y auditoría.
- **Almacenamiento Temporal (/tmp, Parquet):** contenedor temporal para archivos intermedios (parquet) usados durante el preprocesado y reentrenamiento. Facilita transferencia eficiente entre pasos y evita llamadas reiteradas a la DB durante procesamiento.
- **Logging (archivo rotativo, env vars):** captura eventos, métricas y errores. Permite configurar fichero y nivel mediante variables de entorno (`LOG_FILE`, `LOG_LEVEL`). Prefect y la API escriben en este sistema para auditoría y debugging.

5. LIMITACIONES Y TRABAJO FUTURO

Si bien el enfoque propuesto mejora la interpretabilidad de modelos complejos, existen limitaciones importantes que deben reconocerse:

- **Alcance de la metodología:** las técnicas desarrolladas se han probado principalmente en arquitecturas feedforward y sobre datasets tabulares; su aplicabilidad a redes convolucionales o recurrentes no es directa y requerirá adaptaciones metodológicas.

- **Dependencia de las features:** la interpretabilidad y la validez de reglas extraídas dependen fuertemente de la calidad, estabilidad y representatividad de las features. Cambios en el pipeline de preprocesado pueden invalidar reglas previamente extraídas.
- **Riesgos del oversampling:** el uso de SMOTE y técnicas similares introduce ejemplos sintéticos que pueden sesgar métricas o introducir artefactos si no se controlan adecuadamente; es necesario validar que las generalizaciones permanecen válidas en datos reales.
- **Explicaciones local vs global:** el método híbrido propuesto combina señales globales (similitud de pesos) y reglas locales; puede existir tensión entre explicaciones fieles a instancias y explicaciones globales compactas.
- **Monitoreo y deriva:** la estabilidad en producción requiere monitoreo de deriva covariante y de rendimiento, así como mecanismos de reentrenamiento y validación continua.

Trabajo futuro recomendado:

- Extender y evaluar el método en arquitecturas CNN/RNN y en tareas no tabulares (imágenes, series temporales).
- Integrar estimaciones de incertidumbre y calibración de probabilidades para mejorar la confianza operativa de las explicaciones.
- Automatizar pipelines de validación post-despliegue que detecten rupturas en las reglas extraídas y disparen alertas o reentrenamientos.
- Evaluar impacto en equidad y sesgos, incluyendo métricas de fairness y mitigaciones cuando se detecten disparidades.

6. TRABAJOS RELACIONADOS

La literatura sobre explicabilidad y extracción de reglas es amplia. A modo de resumen:

- Métodos de explicación local y global como LIME (Ribeiro et al., 2016) y SHAP (Lundberg & Lee, 2017) que aproximan la conducta de modelos complejos por modelos explicativos locales o por contribuciones de features.
- Modelos sustitutos y extracción de reglas: enfoques que usan árboles de decisión o técnicas de regresión simbólica para aproximar la decisión de redes neuronales y extraer reglas legibles.
- Hibridación simbólico–numérica: trabajos recientes exploran combinar representaciones numéricas con razonamiento simbólico para obtener explicaciones estructuradas y verificables.

Nuestro trabajo se sitúa en la intersección entre la extracción de reglas y las métricas de interpretabilidad basadas en similitud de parámetros, aportando una métrica que combina similitud del coseno con medidas de entropía para priorizar reglas compactas y estables.

7. CONTRIBUCIONES PRINCIPALES

Las contribuciones principales de este trabajo son:

- Propuesta de un algoritmo híbrido simbólico–numérico para extraer reglas interpretables a partir de pesos y activaciones de redes neuronales.
- Definición de una métrica de interpretabilidad que combina similitud del coseno con medidas de entropía para priorizar reglas robustas y compactas.
- Implementación reproducible del pipeline completo, incluyendo preprocesado, oversampling (SMOTE), entrenamiento y despliegue del modelo con API de inferencia documentada.
- Evaluación empírica y conjunto de artefactos (código y modelo serializado) que facilitan la reproducibilidad y la auditoría del proceso.

8. CONCLUSIÓN

Este trabajo demuestra que la combinación de medidas geométricas sobre parámetros (similitud del coseno) con técnicas de razonamiento lógico permite extraer reglas que aumentan la interpretabilidad de modelos densos sin sacrificar en exceso la capacidad predictiva. La implementación práctica (pipeline reproducible y API de inferencia) muestra cómo llevar estas técnicas a un entorno productivo. Para la adopción industrial recomendamos políticas claras de monitoreo, validación post-despliegue y evaluación de impacto en sesgos y equidad.

REFERENCES

- [1] Rumelhart, D.E., Hinton, G.E., Williams, R.J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536.
- [2] Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep Learning*. MIT Press.